

AI FAQ Assistant API

2. Requirement Analysis Phase

1. Purpose

This document captures the functional and non-functional requirements for the AI FAQ Assistant API. It translates the concept approved during the Brainstorming and Ideation phase into a precise, testable set of requirements that will guide system design, development, and testing.

2. Scope of Requirement Analysis

Requirement analysis for this project covers the core question-answering API, FAQ content management, administrator authentication, query logging, and user feedback collection. It excludes advanced features such as multi-language support and voice input, which are documented as future scope rather than requirements for the first release.

3. Stakeholders

Stakeholder	Interest in the System
Project team / developers	Need clear, unambiguous requirements to design and build correctly
End users	Need fast, accurate answers to natural-language questions
Content administrators	Need an easy way to manage FAQ content
Client applications	Need a stable, well-documented API contract
Project supervisor / evaluator	Needs evidence that requirements are complete and traceable

4. Requirement Gathering Methods

Requirements were derived using a combination of methods appropriate for this project's scale:

- Analysis of the problem statement and ideas finalized during brainstorming
- Study of similar existing FAQ/chatbot systems to identify common expected features
- Discussion within the team to define what a minimum viable version must include
- Review of standard API design and NLP system practices from technical documentation

5. Functional Requirements

Functional requirements describe what the system must do. Each requirement is uniquely identified for traceability into design and test documents.

ID	Requirement	Priority
FR-1	The system shall accept a user question via a REST API endpoint (e.g., POST /api/ask).	High
FR-2	The system shall match the question to the most relevant FAQ(s) using NLP-based semantic similarity.	High
FR-3	The system shall return an answer along with a numeric confidence score.	High
FR-4	The system shall return a polite fallback response when no confident match is found.	High
FR-5	The system shall allow authorized admins to create new FAQ entries.	High
FR-6	The system shall allow authorized admins to update existing FAQ entries.	High
FR-7	The system shall allow authorized admins to delete FAQ entries.	Medium
FR-8	The system shall support organizing FAQs into categories or tags.	Medium
FR-9	The system shall log every incoming query along with the matched result and score.	High
FR-10	The system shall allow end users to submit feedback (helpful / not helpful) on an answer.	Medium
FR-11	The system shall require authentication for all admin-only endpoints.	High
FR-12	The system shall provide a way to list and search existing FAQs for admin review.	Medium

6. Non-Functional Requirements

Non-functional requirements describe how well the system must perform its functions, rather than what it does.

6.1 Performance

The API should return a response within approximately 1–2 seconds under normal load, so that the assistant feels instantaneous to end users.

6.2 Scalability

The system should be able to accommodate a growing number of FAQ entries and an increasing number of concurrent requests without requiring a redesign of the core architecture.

6.3 Availability

The API should target an uptime of 99% or higher, as it may be embedded in customer-facing applications where downtime directly affects user experience.

6.4 Security

Admin endpoints must be protected using authentication and authorization so that only permitted users can modify FAQ content. User input must be validated to prevent injection attacks.

6.5 Maintainability

The codebase should be modular, so that the NLP matching engine, database layer, and API layer can each be updated or replaced independently without breaking the others.

6.6 Usability

The API should be simple and well documented (for example, using Swagger/OpenAPI), so that developers integrating it can understand available endpoints without extensive support.

6.7 Reliability

The system should handle unexpected or malformed input gracefully, returning clear error messages instead of failing silently or crashing.

7. Data Requirements

Data Entity	Description
FAQ	Question text, answer text, category, keywords, and internal embedding vector
Category	Grouping label used to organize FAQs (e.g., Billing, Account)
Query Log	Record of each incoming question, the matched FAQ (if any), confidence score, and timestamp
Feedback	Record linking a query log entry to a helpful/not-helpful rating and optional comment
User / Admin Account	Credentials and role information used for authentication

8. Hardware and Software Requirements

Category	Requirement
Development machine	Standard laptop/desktop with at least 8GB RAM for local development
Server (deployment)	Cloud VM or container host, minimum 2 vCPU / 4GB RAM for pilot scale

Database	Relational database such as MySQL or PostgreSQL
Backend runtime	Node.js or Python (with a framework such as Express, Flask, or FastAPI)
NLP component	Sentence-embedding library or third-party AI API for semantic matching
API testing tools	Postman and/or Swagger UI for manual and documented testing
Version control	Git, hosted on a platform such as GitHub

9. Assumptions

- The initial release supports English-language queries only
- FAQ content is curated and entered by administrators, not generated automatically from unstructured documents
- The system will be accessed primarily through API calls rather than a dedicated end-user interface built by this project
- A reasonably sized starter FAQ dataset will be available before testing begins

10. Constraints

- Project timeline and team size limit the scope to the features listed as Must-have and Should-have
- Budget constraints favor open-source NLP tools over paid, high-volume AI services
- The system must be deployable on modest infrastructure suitable for a pilot or academic project

11. Use Case Descriptions

11.1 Use Case: Ask a Question

Field	Details
Actor	End user / client application
Precondition	FAQ database contains at least one active FAQ entry
Main Flow	User submits a question; system returns the best-matching answer with a confidence score
Alternate Flow	If no match exceeds the confidence threshold, a fallback message is returned
Postcondition	The query and its result are logged for analytics

11.2 Use Case: Manage FAQ Content

Field	Details
Actor	Administrator
Precondition	Administrator is authenticated with a valid token
Main Flow	Administrator creates, updates, or deletes an FAQ entry

Alternate Flow	Request is rejected if required fields are missing or invalid
Postcondition	FAQ database reflects the updated content immediately

12. Acceptance Criteria (Sample)

Acceptance criteria define the conditions under which a requirement is considered satisfied. A sample set is shown below for key requirements.

Requirement	Acceptance Criteria
FR-2 (semantic matching)	A paraphrased test question correctly returns the intended FAQ in at least 8 out of 10 sample cases
FR-4 (fallback handling)	A clearly unrelated question never returns a low-quality FAQ as if it were a confident match
FR-11 (admin authentication)	Any request to an admin endpoint without a valid token is rejected with an appropriate error code

13. Requirement Prioritization (MoSCoW)

Category	Requirements Included
Must have	FR-1, FR-2, FR-3, FR-4, FR-5, FR-6, FR-9, FR-11
Should have	FR-7, FR-8, FR-10, FR-12
Could have	Basic analytics dashboard UI
Won't have (this release)	Multi-language support, voice input

14. Requirement Traceability Note

Each functional requirement identifier (FR-1 through FR-12) introduced in this document will be referenced again in the Design, Development, and Testing phase documents, ensuring that every requirement is designed, implemented, and verified before project sign-off.

15. Requirement Validation Approach

Before requirements were finalized, they were reviewed against three simple checks to reduce ambiguity and rework in later phases:

- Clarity — is the requirement stated in a way that two different readers would interpret the same way?
- Testability — can a concrete test case be written to verify this requirement is met?
- Consistency — does this requirement conflict with any other stated requirement?

Requirements that failed any of these checks during internal review were reworded before being included in this document. For example, an early draft requirement 'the system should answer quickly' was refined into the testable non-functional requirement in Section 6.1, which specifies an approximate response-time target.

16. Glossary of Terms Used in This Document

Term	Meaning
Functional Requirement	A statement of what the system must do
Non-Functional Requirement	A statement of how well the system must perform a function
Confidence Threshold	The minimum similarity score required before an FAQ answer is returned
Stakeholder	Any person or group with an interest in, or influence over, the system